

Reversible Database Watermarking Based on Order-preserving Encryption for Data Sharing

DONGHUI HU, QING WANG, SONG YAN, XIAOJUN LIU, MENG LI, and SHULI ZHENG,

Key Laboratory of Knowledge Engineering with Big Data (Hefei University of Technology), Ministry of Education; School of Computer Science and Information Engineering, Hefei University of Technology, Hefei, Anhui, China; Anhui Province Key Laboratory of Industry Safety and Emergency Technology; Intelligent Interconnected Systems Laboratory of Anhui Province (Hefei University of Technology), China

In the era of big data, data sharing not only boosts the economy of the world but also brings about problems of privacy disclosure and copyright infringement. The collected data may contain users' sensitive information; thus, privacy protection should be applied to the data prior to them being shared. Moreover, the shared data may be re-shared to third parties without the consent or awareness of the original data providers. Therefore, there is an urgent need for copyright tracking. There are few works satisfying the requirements of both privacy protection and copyright tracking. The main challenge is how to protect the shared data and realize copyright tracking while not undermining the utility of the data. In this article, we propose a novel solution of a reversible database watermarking scheme based on order-preserving encryption. First, we encrypt the data using order-preserving encryption and adjust an encryption parameter within an appropriate interval to generate a ciphertext with redundant space. Then, we leverage the redundant space to embed robust reversible watermarking. We adopt grouping and K-means to improve the embedding capacity and the robustness of the watermark. Formal theoretical analysis proves that the proposed scheme guarantees correctness and security. Results of extensive experiments show that OPEW has 100% data utility, and the robustness and efficiency of OPEW are better than existing works.

CCS Concepts: • **Security and privacy** → **Management and querying of encrypted data; Privacy protections;**

Additional Key Words and Phrases: Data sharing, security, copyright, watermark, order-preserving encryption, security, data utility, robustness, efficiency

The work described in this article was supported by National Natural Science Foundation of China under the grant No. U1836102 and No. 62002094, Anhui Science and Technology Key Special Program 201903a05020016, Anhui Provincial Natural Science Foundation under the grant No. 2008085MF196, and Anhui Provincial Key Research and Development Program under Grant No. 202104d07020001.

Authors' address: D. Hu, Q. Wang, S. Yan, X. Liu, M. Li (corresponding author), and S. Zheng, Key Laboratory of Knowledge Engineering with Big Data (Hefei University of Technology), Ministry of Education; School of Computer Science and Information Engineering, Hefei University of Technology, 230009 Hefei, Anhui, China; Anhui Province Key Laboratory of Industry Safety and Emergency Technology; Intelligent Interconnected Systems Laboratory of Anhui Province (Hefei University of Technology), Hefei, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

0362-5915/2023/05-ART5 \$15.00

<https://doi.org/10.1145/3589761>

ACM Reference format:

Donghui Hu, Qing Wang, Song Yan, Xiaojun Liu, Meng Li, and Shuli Zheng. 2023. Reversible Database Watermarking Based on Order-preserving Encryption for Data Sharing. *ACM Trans. Datab. Syst.* 48, 2, Article 5 (May 2023), 25 pages.
<https://doi.org/10.1145/3589761>

1 INTRODUCTION

With the development of network and communication technologies, the world has entered the era of big data. The data-driven-based statistics and machine learning technologies further call for large quantities of data. Moreover, companies and governments around the world encourage the sharing and trading of data to help economies boom. However, sharing and trading of data also involve privacy [10, 29, 30] and copyright protection problems [14, 20].

Privacy disclosure accidents in data sharing and trading are frequently reported. For example, in 2017, a famous China big data company was reported to have illegally sold more than 130 million pieces of personal information per day in an eight-month period to obtain profits.¹ In 2018, a data analysis company called Cambridge Analytica collected the personal information of 50 million Facebook users through an APP that detailed the user's personality, social network, and engagement on the platform.² Ultimately, Cambridge Analytics not only used the users' data at will but also violated the privacy of the users, which seriously violated the interests of data copyright holders.

In response to the above privacy leaks, some researchers have proposed solutions from the perspective of encryption. Ordinary encryption may seriously damage the availability of data, whereas **order-preserving encryption (OPE)** was proposed to ensure that data can be sorted and compared after encryption. The earliest OPE was proposed by Argawal et al. in 2004 [2]. Boldyreva et al. proposed **indistinguishability under ordered chosen plaintext attack (IND-OCPA)** in 2009 to make OPE more theoretically complete [5]. Since then, researchers have presented a series of improvement measures for OPE from the perspective of security and efficiency [6, 42]. Since IND-OCPA is ideally secure, it is very hard to achieve. Boldyreva et al. proposed a more realistic definition of the **random order-preserving function (ROPF)** [6]. James Dyer et al. proposed a very efficient high-precision encryption scheme, in which the encryption and decryption process only needs $O(1)$ [13].

In view of the aforementioned data copyright protection issues in data sharing and trading, solutions from the perspective of robust database watermarking have been investigated. The earliest database watermarking method was proposed by Argawal et al. in 2002, but the method cannot embed practical watermark information [1]. Subsequently, Sion et al. proposed a database watermarking scheme embedding meaningful copyright information [41]. Focusing on watermark attacking, people have designed watermark schemes that are robust against malicious attacks such as data modification and deletion [21]. While database watermarking technology has been rapidly developed due to its imperceptibility and security, it encounters the drawback of unrecoverable data after embedding with a watermark. To solve this, reversible watermarking techniques are proposed to recover the original data. In 2006, a reversible database watermarking scheme was implemented based on **differential expansion (DE)** and **histogram shift (HS)** [49]. In 2010, Farfoura used **predictive-error expansion (PEE)** to implement a reversible watermarking scheme and improved the embedding rate, but the watermark remained fragile [15]. In 2017, Franco used

¹<https://finance.sina.com.cn/spread/thirdmarket/2018-07-10/doc-ihezpzwu8601594.shtml>.

²<https://www.reuters.com/article/us-facebook-privacy/facebook-says-data-leak-hits-87-million-users-widening-privacy-scandal-idUSKCN1HB2CM>.

the extension of HS to propose a reversible watermarking scheme with low data distortion, but it has a low embedding capacity [17].

Encryption (such as OPE) technology and database watermarking technology address the privacy and copyright protection problems, respectively. However, determining how to design a method to solve both the privacy and copyright protection problems at the same time remains an urgent research challenge. Combining the two types of technologies, some works have been proposed. In 2005, Bertino proposed a framework that combines k-anonymity technology with digital watermarking technology to achieve both privacy and copyright protection. However, this method makes the data more generalized and incurs low utility of the data and a low embedding rate [3]. In 2011, Zhang used stream encryption to compress the entire image and then embed the watermark [47]. Nevertheless, encryption and watermark embedding are inseparable processes. The embedding rate of watermarks is relatively low, and the process can be completely reversible under certain conditions. Immediately thereafter, Zhang proposed a watermarking method with a separable encryption domain so the watermark can be extracted without first decrypting [48]. However, the watermark embedding rate of this method is relatively low, and the process can be completely reversible under certain conditions. Moreover, the methods of References [47] and [48] lie in the digital image domain. In 2018, a database watermarking method based on order-preserving encryption was also proposed, but it mainly embeds a fragile watermark into the images stored in the database [46]. However, the watermarking method proposed in Reference [46] belongs to the traditional digital image watermarking methods; moreover, the proposed watermarking is a fragile watermark used to detect whether the data have been tampered with, but it cannot be used for robust copyright tracking.

In this article, we propose a new solution to provide both privacy protection and copyright protection (we call it **OPEW, Order-Preserving Encryption-based Watermarking**). However, several challenges still exist. The first challenge is how to choose a suitable privacy protection method. Both generalization strategies and stream encryption severely reduce the availability of encrypted data. Fully homomorphic encryption [19, 23, 27, 35] may be a good solution to this issue, but full homomorphic encryption has the problem of an excessive computational cost. Recent researches [11, 12, 40] are still $10,000\times$ to $100,000\times$ slower than unencrypted computation when executed in carefully optimized software. Therefore, in this article, we use the order-preserving encryption scheme, which has a low computational cost. The order-preserving encryption scheme can perform operations such as comparison and sorting in the ciphertext domain and provide ciphertext data management for the user.

The second challenge is to embed a watermark without destroying the security of encryption. We design a database watermarking scheme based on a state-of-the-art OPE scheme [13]. The watermark information is embedded without destroying the nature of OPE. Whether the watermark information is extracted before decryption or not should not destroy the order-preserving characteristics.

The third challenge is that the embedding process of the designed watermarking scheme may lead to distortion of the original data. For some data that cannot tolerate data deviation, such as any small distortion in the year of birth, the process will cause a large deviation, resulting in reduced data utility. Consequently, when choosing how to embed the watermark into the encrypted ciphertext without causing distortion of plaintext, lossless watermarking (in which the embedded watermark does not affect the original data) [39] provides a good solution. We design a lossless embedding watermarking method based on order-preserving encryption. The embedding of watermarks in the ciphertext data does not affect the plaintext data from being decrypted correctly.

The contributions of the proposed methods include the following three aspects:

- We propose a lossless watermarking scheme based on order-preserving encryption, which can provide both privacy protection and copyright tracking. We first combine OPE and robust database watermarking on the database. The watermark can be extracted from the embedded data to prove the copyright of the ownership of the data. The embedded data can be decrypted to obtain the original data, i.e., lossless recovery.
- We design a K-means-based data preprocessing method to increase the relative embedding capacity of the embedded watermark and thus enhance the robustness of the watermark database.
- We formally prove the correctness and security of OPEW. Extensive experiments show that our scheme exhibits high robustness in defending against watermark attacks, high data utility of the recovered data, a high embedding capacity, and a low computational cost in copyright protection on three databases.

The rest of this article is organized as follows: The background information on OPE and reversible database watermarking is given in Section 2. The system model, threat model, and design objects are shown in Section 3. The preliminaries in our article are reviewed in Section 4. The algorithm frame and details of OPEW are described in Section 5. The order-preserving property, correctness, and security of OPEW are proved in Section 6. The performance analysis of OPEW is presented in Section 7. The discussion is given in Section 8. In Section 9, we conclude our work and present some perspectives on future directions.

2 RELATED WORK

Our method is a combination of encryption and watermarking scheme. Here, we will briefly introduce works relevant to this study, including OPE and reversible database watermarking.

2.1 Order-preserving Encryption

We assume that an encryption scheme uses a key for two arbitrary numbers in the plaintext domain X_i and X_j ; the corresponding ciphertexts are Y_i and Y_j . If $X_i < X_j \Rightarrow Y_i < Y_j$ always holds, then we say that this scheme is an OPE scheme.

In 2004, Agrawal et al. [2] proposed an OPE scheme that randomly extracts q data from a dataset provided by the user and sorts the q data to obtain a key table T . Then, the mapping relationship is established such that the encrypted ciphertext of the i th data point c_i in the database is $d_i = T[i]$. Since the data in the key table are sorted, the ciphertext retains the order of the plaintext. When decrypting, as long as d_i is the first number in the key table, it can be decrypted. However, this method also has certain defects. First, the key is generated according to the dataset used; second, the updating operation of the dataset cannot be supported.

In 2009, Blodyreva et al. [5] improved the security of OPE and provided a security definition. Their specific method is as follows: M consecutive integers of the plaintext domain are mapped to integers in a larger range $[1:N]$, and the range is then recursively divided into M monotonically increasing subranges to construct random OPE. The algorithm then recursively divides the range into two subranges and performs sampling from the range field during each recursion until the sampled value and the input plaintext value are equal.

Subsequently, OPE has been continuously developed, which has quickly attracted the attention of researchers and furthered its rapid development [7, 18]. In Reference [7], Boehn et al. proposed an OPE scheme that is a new type of encryption scheme suitable for range queries. It can determine the size of the plaintext from the ciphertext size. In Reference [18], Furukawa constructed the first encryption scheme that is visible in the order of the trap gates. It requires that one must know

the label of one party when comparing the size of two encrypted ciphertexts. In 2017, Dyer et al. proposed a simple, efficient, and computationally intensive OPE scheme [13]; a random sequential order-sequence encryption scheme based on the **general approximate common divisor problem (GACDP)** was proposed.

2.2 Reversible Database Watermark

Reversible watermarking technology first appeared in the field of images. In 2003, Tian and Jun [44] pioneered the **differential expansion (DE)** algorithm with a high embedding capacity, but the robustness is poor. In 2006, Ni et al. [38] proposed another milestone algorithm based on **histogram translation (Hs)**. This method is simple and easy to understand. The embedded watermark has a weaker distortion effect on the data, better robustness, but insufficient embedding capacity. As related concepts and technologies became sufficiently mature, watermarking technology was rapidly applied to the database field. Here, we will present a detailed introduction to the database watermarking technology that combines HS and DE as an example.

In 2008, Gupta and Pieprzyk proposed differential extended-based reversible **database watermarking (DEW)** [22], which uses differential extension to perform a reversible arithmetic operation on any two attribute values in a tuple to embed a watermark bit. In 2010, M. E. Farfoura et al. proposed a **predicting error expansion-based reversible database watermarking (PEEW)** method [16]. PEEW reduces data distortion by embedding a watermark into the fractional part of the attribute value and then uses a prediction function to obtain a prediction value. Next, PEEW subtracts all the values in each group from the predicted values to obtain a deviation and embeds the watermark information into the deviation to obtain a new deviation.

In Reference [25], Hu proposed a reversible database watermarking technique based on a **GA and histogram shifting differential extension (GAHSW)**. In GAHSW, the data is preprocessed with a GA to minimize the data distortion and simultaneously improve the watermark robustness. The database is grouped by using the primary key of the tuple and the watermarking key. After grouping the data, GAHSW uses the idea of DE to construct a “reference point” (the average of the maximum and minimum values), calculates the difference between each point in the group and the “reference point,” and uses the prediction error to construct a histogram. The HS correlation technique is then used to move left and right at the peak points (p and $-p$) of the histogram to create new vacancies, thereby embedding the watermark with large capacity and good robustness. However, the GA process is time-costing, which may have time performance bottleneck. In addition, DEW, PEEW, and GAHSW are pure watermarking schemes, whereas our method also provides privacy protection.

3 PROBLEM STATEMENT

3.1 System Model

Consider a system model involving four different entities: the data owner, the cloud server, the potential data user, and the long-term data user, as depicted in Figure 1.

- **Data owner:** To protect the data privacy without loss of usability, the data owner encrypts the dataset with OPE. Due to the nature of OPE, encrypted data can be compared and classified. Then, the data owner uses the redundant space generated by the OPE encryption to perform the preprocessing operation and finally embeds the watermark information losslessly. We modify a random number used in OPE without reducing the security of OPE. We then use this free space to embed the watermark information so it returns to the previous range. The encrypted watermarked dataset is uploaded to the cloud server and published for data sharing or trading.

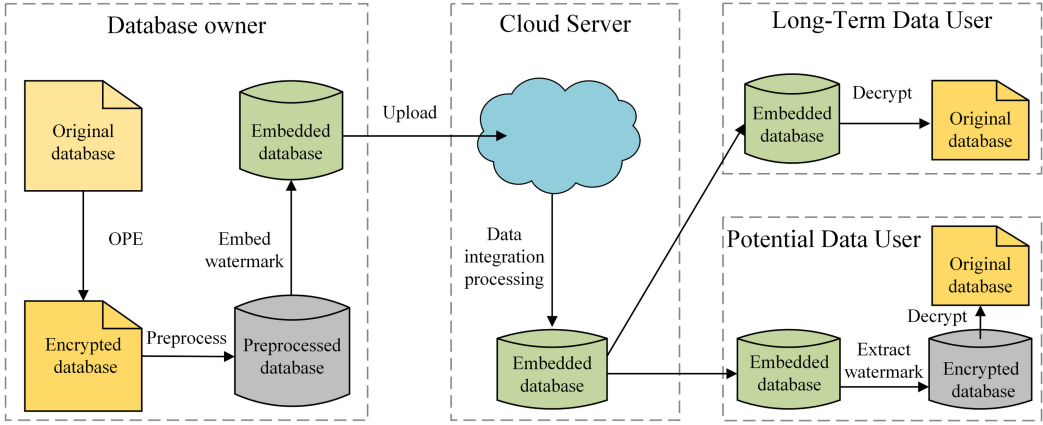


Fig. 1. Lossless database watermarking scheme based on order-preserving encryption.

- **Cloud server:** In this scenario, the cloud server stores data for data owners and returns partial data for the data user.
- **Potential data user:** The potential data user asks to check the copyright of data before purchasing it. Therefore, he/she will access the watermark key from the data owner. If the potential data user confirms the copyright of the data, then he/she purchases the decryption key from the data owner. The potential data user decrypts the extracted data and obtains the original data.
- **Long-term data user:** The long-term data user trusts the data source. He/she will request the encryption key from the data owner. The long-term data user decrypts the embedded data to obtain the original data.

3.2 Threat Model

Similar to (z, n) -window one-wayness model in Reference [6], we define (z, n) -window one-wayness for an order-preserving encryption-based watermarking scheme. Since an adversary can collect many data in real life, we give the adversary multiple embedded watermark data as challenge in model. We actually consider the more universal data security, which requires that the adversary guess an interval (window) within which the underlying challenge plaintext lies. Therefore, the security model facilitates a better awareness of how accurately the adversary can identify the position of a value.

3.3 Security, Data Utility, Robustness, Efficiency Objectives

There are four objectives for designing OPEW, including security, data utility, robustness, efficiency.

Security. Given multiple embedded data, any adversary cannot locate the range of original data with polynomial-time algorithm.

Data Utility. It includes two aspects: The embedded data should preserve the order of encrypted data, and the decrypted data should be lossless.

Robustness. The extracted watermark should be integrity-preserving under the database modification, deletion, and addition attacks.

Efficiency. OPEW scheme should achieve good performance with respect to computational cost.

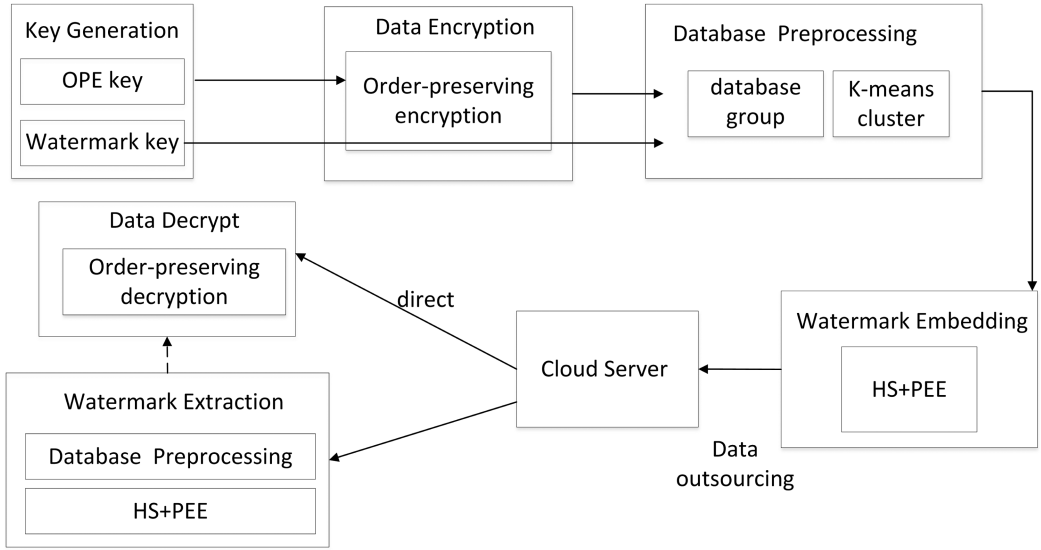


Fig. 2. Flow chart of the proposed encrypted domain database watermarking scheme.

4 PRELIMINARIES

4.1 Order-preserving Encryption

A order-preserving encryption consists of three algorithms: KGen, Enc, and Dec.

- $KGen(1^\lambda)$: This algorithm takes the security parameter λ as input and outputs the encryption key K .
- $Enc(m, K)$: This algorithm takes the encryption key K and the plaintext m as inputs and outputs the corresponding ciphertext c .
- $Dec(c, K)$: This algorithm takes the ciphertext c as input and outputs the corresponding plaintext m .

The correctness of OPE scheme requires that $m = Dec(Enc(m, K), K)$. In addition, the order-preserving property of OPE scheme requires that $m_1 \leq m_2$ if and only if $Enc(m_1, K) \leq Enc(m_2, K)$.

5 THE PROPOSED SCHEME

Before diving into technical details, we first introduce the architecture of our scheme.

5.1 Scheme Overview

The OPEW system consists of six algorithms, as shown below:

- **Key Generation:** $KGen(1^\lambda)$. The algorithm takes a security parameter λ as input and outputs an encryption key K for OPE and a watermark key K_s for database preprocessing.
- **Data Encryption:** $DEnc(D, K)$. The algorithm takes the database D as input. It conducts an interval shrink operation on OPE to create redundant space for watermark embedding and encrypts the database D using the OPE to obtain the encrypted database X .
- **Data Preprocessing:** $DPre(X, K_s)$. The algorithm takes the encrypted database X and K_s as inputs. It divides X into several groups $G = \{G_0, \dots, G_{N_g-1}\}$ with a hash function. And for each group, it further clusters each groups to subgroups with K-means technology. At last, it outputs the preprocessed database X . The algorithm enhances the capacity and robustness of embedded watermark in a certain degree.

Table 1. Key Notations

<u>Indices</u>	
D	the original database
X	the encrypted database
EM	the embedded database
W	watermark information
K	encryption key for OPE
K_s	watermark key for grouping
$tu.PK$	the primary key of the tuple, i.e., the unique identifier for tuple
N_g	the number of groups that generally equals the length of the watermark
N_a	the number of attributes
A_l	the l th attribute column of M
G_i	the i th the group
$G_{i,j}$	the j th subgroup in G_i
$G_{i,j}[l]$	the intersection of A_l with $G_{i,j}$
<u>Parameters</u>	
m	the plaintext
x	the encrypted data
$x'_{i,j}[l]$	the predicted value for $G_{i,j}[l]$
$x''_{i,j}$	the embedded data
p^e	the prediction error
p^{ne}	the normalized error
p'^e	the correction error
p	a peak point that is the horizontal axis with the maximum frequency in the histogram

- **Watermark Embedding:** $WEmb(X, W)$. The algorithm takes the preprocessed X and watermark information W as inputs. For $i \in [0, N_g - 1]$, it embeds $W[i]$ to all subgroups in G_i using **histogram shift (HS)**, and Farfoura et al. used predictive-error Expansion PEE. It outputs the embedded database EM . Due to the interval shrink operation of *OPE*, EM will not disturb the order of X .
- **Watermark Extraction:** $WExt(EM, K_s)$. The algorithm takes EM and K_s as inputs. It extracts the watermark from EM and recovers the encrypted database X . It outputs W and X .
- **Data Decryption:** $DDec(X/EM, K)$. The algorithm takes X or EM and K as inputs. It decrypts X using the decryption algorithm of *OPE* to obtain the original database. Also, it obtains the original database by decrypting EM directly. In short, it takes D as output.

To clarify our scheme further, we show the main phases of our scheme in Figure 2 and key notations used in our article in Table 1. First, the data owner runs algorithm *KGen* to generate keys K and K_s . For security, the data owner encrypts D using algorithm *DEnc*. Second, the data owner divides the database into several subgroups by using algorithm *DPre*. For measuring database robustness, the data user embeds watermarks to the preprocessed database and uploads it to cloud server. Likewise, the data user extracts the watermark by algorithm *DExc*. And if the data user wants the original database, he/she can decrypt the encrypted database obtained by algorithm *DDec* or decrypt the embedded database directly.

5.2 Key Generation

The order-preserving encryption scheme used for data encryption is a symmetric encryption scheme whose range of plaintext fields is M , and the scope of the ciphertext fields is N . First, we need to choose a key K ; however, the choice of this key K is associated with the security

parameter λ . This security parameter λ is a conditional constraint satisfying $\lambda > \frac{8}{3} \lg M$. Then, we randomly select a value in the range of $[2^\lambda, 2^{\lambda+1}]$ as the key K .

For database preprocessing, we choose a hash function $H : \{0, 1\}^* \rightarrow Z_{N_g}$ and watermark key $K_s \in \{0, 1\}^w$ for it.

5.3 Data Encryption

Order preserving encryption (OPE) is a special encryption scheme that preserves the order of plaintext in ciphertext. It can not only protect data privacy, but also enable efficient query on encrypted data.

The specific encryption steps are as follows:

$$x = \text{Enc}(m, K) = m * K + r, \quad (1)$$

where $r \xleftarrow{\$} (K^{\frac{3}{4}} + 1, K - K^{\frac{3}{4}} - 1)$ ($\$$ means arbitrarily select one from the following range), and m and x are the plaintext and ciphertext, respectively. The security of the algorithm is based on the difficulty of solving the general approximate common divisor problem [13]. The whole process of OPE is relatively simple. The computational cost of the encryption and decryption process is $O(1)$.

The watermark embedding algorithm is to embed watermarks into the encrypted database. OPE provides a redundant space to tolerate the change. Specifically, the original constraint in Reference [13] on r is $r \xleftarrow{\$} (K^{\frac{3}{4}}, K - K^{\frac{3}{4}})$. The watermark embedding will shift the ciphertext x to $x'' = x \pm 1$ or remains unchanged. We shrink the constraint interval to $(K^{\frac{3}{4}} + 1, K - K^{\frac{3}{4}} - 1)$ to support for $r \pm 1$ is in the original interval. Therefore, there exists enough redundant space to support the watermark embedding. In fact, the size of the redundant space is $(K^{\frac{3}{4}} + t, K - K^{\frac{3}{4}} - t)$, $t \in [1, \lfloor \frac{K}{2} - K^{\frac{3}{4}} \rfloor]$. When $t = 1$, the size of the redundant space is maximum. In our article, we use $t = 1$.

5.4 Database Preprocessing

In database watermarking, grouping is the most common preprocessing method. Grouping is mainly for data secret sorting or the tuple combination operation. The use of grouping operations can improve the robustness of the watermarking. While there are many ways to group, the most common of which is:

$$n_u = H(K_s | H(K_s | t_u \cdot PK)) \bmod N_g, \quad (2)$$

where n_u indicates the group number of the tuple, H is the cryptography hash function (such as SHA-1 and MD5), K_s is the database watermarking key, $|$ indicates the bit connection operation, $t_u \cdot PK$ is the primary key of the tuple, and N_g is the number of groups to be divided (which generally equals the length of the watermark information).

After the database has been grouped into N_g groups, each group contains the encrypted attribute values. The elements in the group are scattered, which also means that the embedding capability is low. Embedding rate is EC/N_n , EC means embedding capability, and N_n means the number of embedded positions. To improve the embedding rate of watermarks, we need to improve the strategy. PEE [16] is a good method to solve this problem. PEE found initial use in the field of digital image watermarking and mainly uses the local correlation of the image. In a natural image, the difference between each pixel and its surrounding pixels is very small, and it can be used to design a prediction function [43] to predict the pixel by its surrounding pixels. Borrowing this idea, we use K-means [28] clustering to achieve similar results in a database.

Our main idea is that we first group all tuples using Equation (2) and then apply K-means to each group to cluster them and obtain several subgroups. K-means is a deterministic algorithm. If the number of clusters (subgroups) k is specified, then generated subgroups are fixed. For each

group, the first step of K-means is to determine k . Second, k encrypted attribute values are chosen as centroids randomly. Then, the distance of all encrypted attribute values in this group from the centroids are calculated, and each encrypted attribute is assigned to the cluster with the closest centroid to it. After that, the average of each cluster is calculated to update the centroid, and the above process is recursively executed until the centroid does not change. Finally, the k subgroups are generated. In the process of K-means, it is difficult to determine how many clusters (k) there are. We first try some suitable values, obtain the cluster centroids by a K-means algorithm, merge the nearest classes according to the distance of the k clusters, and iterate to an appropriate value of k . The selection of k depends on the content of the database. Experiments suggest that the value of k for our database is between 3 and 8. If $k < 3$, then the embedding capacity is less because there are fewer subgroups. If $k > 8$, then the embedding capacity is less because of the low frequency of the peak point. Although K-means is somewhat troublesome in the choice of centroids, its entire process is simple and fast, and it is advantageous in dealing with big data.

The K-means algorithm is important for OPEW. Clustering makes the approximate data cluster into a subgroup. The difference between each data in a subgroup is relatively small, and it is easy to find the peak point. In fact, we embed the watermark at the peak point. Without clustering, the robustness of OPEW will be weak. The reason is that the frequency of the peak point will be low and only one watermark bit can be embedded in one attribute column, so embedded watermark will be less. But it does not affect the data utility. The embedded data preserve the order of encrypted data, and the decrypted data are lossless.

5.5 Watermark Embedding

The main idea of watermark embedding is to first use OPE to encrypt the plaintext and then use the redundant space generated by OPE, which has a certain range of intervals, to embed the watermark. As aforementioned in Section 5.3, we slightly modify the redundant space so the modifications caused by the embedded watermark do not affect its order. We know that the HS-based watermark scheme [38] causes a small modification of the data, but its embedding capacity is not very high. The embedding capacity of the PEE-based watermark embedding scheme [16] is relatively high, but the modification of the data is relatively large. Therefore, we combine the two schemes and take advantage of them. We first use the prediction function to estimate the value of each attribute based on the spatial correlation generated by grouping and clustering. The spatial correlation indicates the relative proximity of data in subgroups after grouping and clustering. The same data will be in a subgroup. When we construct a histogram with the horizontal axis representing the prediction error p^e and the vertical axis representing the frequency of p^e , it enlarges the frequency of the peak point in the histogram and enhances the robustness of the watermark. Then, we calculate the deviation between each actual value and the predicted value. With the error between the actual value (encrypted data) and the predicted value replacing the real value, the frequency of peak point in histogram can be higher, enhancing the robustness. By modifying the prediction error histogram, a large amount of payload can be embedded in the data, and by using both expansion and translation, the embedded distortion can be controlled.

We borrow the idea that predicts the pixel by its surrounding pixels from the digital image reversible watermarking scheme [43]. For each set $G_{i,j}[l]$, $i \in [0, N_g - 1]$, $j \in [0, N_{s-i}]$, $l \in [1, N_a]$ that the intersection of the l th attribute column with the j th subgroup in the i th group, we need to compute predicted value. The prediction function for $G_{i,j}[l]$ is shown in Equation (3), which calculates the average of the maximum and minimum values in $G_{i,j}[l]$:

$$x'_{i,j}[l] = \left\lfloor \frac{\max(G_{i,j}[l]) + \min(G_{i,j}[l])}{2} \right\rfloor. \quad (3)$$

During database preprocessing phase, we cluster the approximate data to a subgroup. The difference between each data in a subgroup is relatively small, and it is very efficient to use the formula to predict the attribute value.

Then, we calculate the prediction error for each element x in $G_{i,j}[l]$ by using the following formula:

$$p^e = x - x'_{i,j}[l]. \quad (4)$$

Next, we use all prediction errors to construct a frequency histogram. Before embedding the watermark, we need to normalize the prediction errors. First, we find the peak point p , which is the horizontal axis with the maximum frequency in the histogram of $G_{i,j}$. Then, we use Equation (5) to compute the normalized errors, so the normalized error corresponding to p is 0:

$$p^{ne} = p^e - p. \quad (5)$$

Now, we can construct a histogram with the horizontal axis representing the normalized error p^{ne} and the vertical axis representing the frequency of the normalized error.

After constructing the histogram, we use Equation (6) to embed the watermark bit w with the error range $[-1, 0]$, and the other interval ranges are extended by one unit to the left or the right.

$$p'^e = \begin{cases} p^{ne} + 1, & p^{ne} > 0; \\ 2 * p^{ne} + 0, & -1 \leq p^{ne} \leq 0, w = 0; \\ 2 * p^{ne} + 1, & -1 \leq p^{ne} \leq 0, w = 1; \\ p^{ne} - 1, & p^{ne} < -1. \end{cases} \quad (6)$$

Finally, based on the correction error p'^e , predicted value $x'_{i,j}[l]$, and peak point p , the embedded attribute value with the embedded data is obtained as:

$$x'' = p'^e + x'_{i,j}[l] + p. \quad (7)$$

We repeat the above process for $G_{i,j}[l]$, $0 \leq i \leq 1$, $0 \leq j \leq N_{s-i} - 1$, $1 \leq l \leq N_a$, where N_{s-i} denotes the number of subgroups of G_i . The specific watermark embedding algorithm is shown in Algorithm 1, which takes an encrypted database X , a watermark key K_s , and watermarks W as inputs and outputs an embedded database EM and array pe . In the algorithm, N_g , N_{s-i} , and N_a are the numbers of groups, subgroups of G_i , and attributes of the database, respectively. We use the array pe to record the peak point of prediction errors.

5.6 Watermark Extraction

After the watermark is embedded, the embedded data can be uploaded to the cloud server and shared with data users. When a potential data user occurs, he/she interacts with the data owner and obtains the watermark key, so he/she can extract watermarks to check the copyright of data. Our first task in the watermark extraction phase is the same as that in the watermark embedding phase. The database is first grouped by using Equation (2) and then clustered by applying K-means on each group. We construct the prediction function using Equation (3) to obtain the predicted values $x'_{i,j}[l]$ s and use the following formula to find the prediction error of each attribute value x'' in a subgroup:

$$p^e = x'' - x'_{i,j}[l]. \quad (8)$$

After obtaining the prediction errors after embedding the watermark, we then perform normalization on the errors:

$$p^{ne} = p^e - p, \quad (9)$$

where p is the peak point of the errors located in array pe .

ALGORITHM 1: Watermark Embedding**Input:** Variables: X, K_s, W .**Output:** EM, pe

```

1: Use Equation (2) to group  $X$  into  $G_0, \dots, G_{N_g-1}$ 
2: for  $i = 0$  to  $N_g - 1$  do
3:   Get one bit of watermark bit  $W[i]$ ;
4:   Use the K-means algorithm to cluster  $G_i$  into  $G_{i,0}, \dots, G_{i,N_{s-i}-1}$ ;
5:   for  $j = 0$  to  $N_{s-i} - 1$  do
6:     for  $l = 1$  to  $N_a$  do
7:       Use Equation (3) to find a predicted value for  $G_{i,j}[l]$ ;
8:       Use Equation (4) to calculate the prediction errors and record the peak point  $p$  in array
          $pe$ ;
9:       Use Equation (5) to obtain the normalized errors;
10:      Use the normalized errors to construct a histogram;
11:      Use Equation (6) to embed the watermark bit  $W[i]$ ;
12:    end for
13:  end for
14: end for
15: return  $EM, pe$ .
```

Finally, we can use Equation (10) to recover the encrypted data. When $-2 \leq p^{ne} < 2$, we extract the watermark bit by Equation (11).

$$x = \begin{cases} x'' - 1, & p^{ne} \geq 2; \\ \left\lfloor \frac{x'' + p + x'_{i,j}[l]}{2} \right\rfloor, & -2 \leq p^{ne} < 2; \\ x'' + 1, & p^{ne} < -2. \end{cases} \quad (10)$$

$$W[i] = \left\lfloor \frac{p^{ne}}{2} \right\rfloor, \quad -2 \leq p^{ne} < 2. \quad (11)$$

The watermark extraction algorithm is illustrated in Algorithm 2, where the “majority voting mechanism” is used to determine the final watermark bit in the i -th group, which means that if the number of $W[i]$ s supporting “1” is larger than the number of $W[i]$ s supporting “0,” then the final extracted watermark bit is “1”; otherwise, it is “0.”

5.7 Data Decryption

OPEW has two decryption methods. The first one is that the potential data user who owns the encryption key decrypts the encrypted data after extracting the watermark. The process is as follows:

$$m = Dec(x, K) = \left\lfloor \frac{x}{K} \right\rfloor. \quad (12)$$

The second one is that the data user also decrypts the embedded data directly. Similarly, the decryption process is as follows:

$$m = Dec(x'', K) = \left\lfloor \frac{x''}{K} \right\rfloor. \quad (13)$$

Table 2. A Group G_i

A1	A2	A3
2	2	2
3	2	4
7	2	8
9	7	9
4	7	1
8	8	2
4	9	8
5	10	10

Table 3. The Processed G_i by K-means

A1	A2	A3
2	2	2
3	2	4
7	2	8
9	7	9
4	7	1
8	8	2
4	9	8
5	10	10

ALGORITHM 2: Watermark Extraction**Input:** Variables: EM, pe, K_s .**Output:** X, W

- 1: Use Equation (2) to group EM into $EM_0, \dots, EM_{N_g-1}$;
- 2: **for** $i = 0$ to $N_g - 1$ **do**
- 3: Use the K-means algorithm to cluster EM_i into $EM_{i,0}, \dots, EM_{i,N_{s-i}-1}$;
- 4: **for** $j = 0$ to N_{s-i} **do**
- 5: **for** $l = 1$ to N_a **do**
- 6: Use Equation (3) to find a predicted value for each $EM_{i,j}[l]$;
- 7: Use Equation (8) to calculate the prediction errors;
- 8: Use Equation (9) to obtain the normalized errors;
- 9: Use Equation (11) to extract the watermark $W[i]$ and use Equation (10) to recover the data;
- 10: **end for**
- 11: **end for**
- 12: Apply the “majority voting mechanism” to group G_i and determine the final watermark bit $W[i]$.
- 13: **end for**
- 14: return X, W .

5.8 An Example

Here, we use a simple example to illustrate the embedding scheme. For the sake of simplicity, we only consider a group and the data are encrypted. As shown in Table 2, G_i has 8 tuples and $N_a = 3$. According to Algorithm 1, we should cluster the 24 values by K-means algorithm in Step 4. For simplicity, we set N_{s-i} to 2, i.e., $k = 2$. The clustered groups are shown in Table 3. $G_{i,0}$ consists of attribute values marked red. $G_{i,1}$ consists of attribute values marked black. In Step 7, we need to calculate the predicted value for $G_{i,0}[1]$. We use $G_{i,0}[1]$ as an example to show the process of embedding a watermark $W[i]$. In this example, $G_{i,0}[1] = [2, 3, 4, 4, 5]$. We obtain the predicted value $x'_{i,0}[1] = 3$ by Equation (3). In Step 8, we use Equation (4) to calculate the prediction errors $[-1, 0, 1, 1, 2]$ and add the peak point 1 in pe . In Step 9, we normalize the prediction errors by Equation (5) and obtain $[-2, -1, 0, 0, 1]$. We can use Equation (6) to calculate the correction error. When $W[i] = 1$, the correction error is $[-3, -1, 1, 1, 2]$. When $W[i] = 0$, the correction error is $[-3, -2, 0, 0, 2]$. The corresponding histograms for the prediction error, normalized error, and correction error are shown in Figure 3. For $W[i] = 1$, we obtain the embedded data $[1, 3, 5, 5, 6]$ based on the correction error, predicted value, and peak point. For $W[i] = 0$, we obtain the embedded data $[1, 2, 4, 4, 6]$. $G_{i,0}[2], G_{i,0}[3], G_{i,1}[1], G_{i,1}[2], G_{i,1}[3]$ follow the same way.

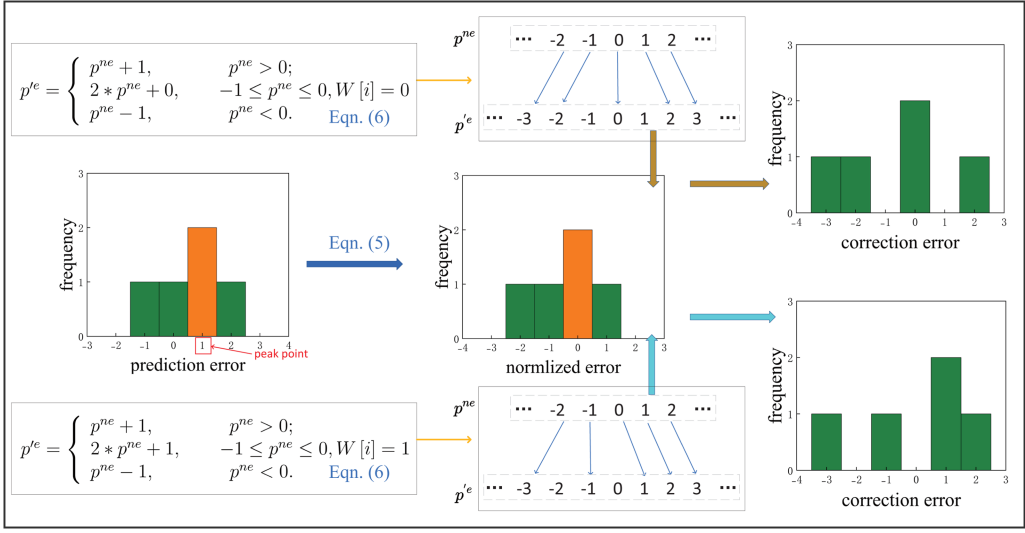


Fig. 3. The histograms for the prediction error, normalized error, and correction error of $G_{i,0}[1]$.

For watermark extraction of the embedded data $[1, 2, 4, 4, 6]$, we first compute predicted value $x'_{i,0}[1]$ by Equation (3). Then, we use Equation (8) to obtain the prediction error $[-2, -1, 1, 1, 3]$. During the watermark embedding phase, we have recorded the peak point 1 in pe . Therefore, we can obtain the normalized errors $[-3, -2, 0, 0, 2]$ according to Equation (9). Based on Equation (11), we can obtain three 0s from $[-2, 0, 0]$, which are the watermarks we have embedded. Based on Equation (10), we can obtain the encrypted data $[2, 3, 4, 4, 5]$. The watermark extraction of $[1, 3, 5, 5, 6]$ follows the same way.

6 CORRECTNESS AND SECURITY ANALYSIS

In this section, we analyze the order-preserving property, correctness, and security of OPEW.

6.1 Order-preserving Property Analysis

Note that in Equation (1), we shrank the constraint interval of OPE to $(K^{\frac{3}{4}} + 1, K - K^{\frac{3}{4}} - 1)$. We prove that OPEW still retains the order-preserving property.

After applying OPE and processing operations, we use the watermarking embedding scheme proposed in Section 5.5 to embed the watermark. Specifically, we use Equation (6) to embed a watermark bit. In the equation, when the embedded watermark bit is 0, the correction error p'^e remains unchanged; otherwise, the correction error p'^e is added to or subtracted by 1. Through Equations (3)–(7), we can easily find that the final watermarked attribute value x'' alters one bit from the original attribute x at most, which results in the constraint interval falling in $(K^{\frac{3}{4}}, K - K^{\frac{3}{4}})$, i.e., the secure constraint interval proven by Howgrave-Graham [24].

We can also prove that the r_i -modified OPEW does not destroy the order-preserving property of OPE. Given two attribute values X_1 and X_2 ($X_1 > X_2$), let Y_1 and Y_2 be the final values after the OPEW process, i.e., $Y_1 = K * X_1 + r_1 \pm 1$ and $Y_2 = K * X_2 + r_2 \pm 1$. To ensure that $Y_1 > Y_2$, we have:

$$\begin{aligned} Y_1 &> Y_2 \\ \Leftrightarrow K * X_1 + r_1 \pm 1 &> K * X_2 + r_2 \pm 1 \\ \Leftrightarrow K * (X_1 - X_2) &> (r_2 \pm 1) - (r_1 \pm 1) \end{aligned}$$

Since $r_1, r_2 \in (K^{\frac{3}{4}} + 1, K - K^{\frac{3}{4}} - 1)$. Let $r'_1 = r_1 \pm 1$ and $r'_2 = r_2 \pm 1$, then $r'_1, r'_2 \in (K^{\frac{3}{4}}, K - K^{\frac{3}{4}})$. We have:

$$Y_1 > Y_2$$

$$\Leftrightarrow K * (X_1 - X_2) > r'_2 - r'_1$$

* **Case 1:** $r'_2 \geq r'_1$.

Since $X_1 - X_2 \geq 1$, $r'_2 - r'_1 \geq 0$ and $r'_2 - r'_1 < (K - K^{\frac{3}{4}}) - K^{\frac{3}{4}} = K - 2K^{\frac{3}{4}}$. We have:

$$K * (X_1 - X_2) \geq K > K - 2K^{\frac{3}{4}} > r'_2 - r'_1 \geq 0$$

$$\Rightarrow K * (X_1 - X_2) > r'_2 - r'_1$$

$$\Rightarrow Y_1 > Y_2$$

* **Case 2:** $r'_2 < r'_1$.

Since $X_1 - X_2 \geq 1$, $r'_2 - r'_1 < 0$ and $r'_2 - r'_1 > K^{\frac{3}{4}} - (K - K^{\frac{3}{4}}) = 2K^{\frac{3}{4}} - K$. We have

$$K * (X_1 - X_2) \geq K > 0 > r'_2 - r'_1 > 2K^{\frac{3}{4}} - K > -K$$

$$\Rightarrow K * (X_1 - X_2) > r'_2 - r'_1$$

$$\Rightarrow Y_1 > Y_2$$

Therefore, our OPEW retains the order-preserving property, i.e., $X_1 > X_2 \Rightarrow Y_1 > Y_2$.

6.2 Correctness Analysis

Data decryption: Assume that the data user only has encryption key K and watermark key K_s . After watermark extraction, the data user obtains encrypted database. He/she decrypts the ciphertext in encrypted database to recover the plaintext. The process is as follows:

$$\begin{aligned} \text{Dec}(x, K) &= \left\lfloor \frac{x}{K} \right\rfloor \\ &= \left\lfloor \frac{m * K + r}{K} \right\rfloor \\ &= \left\lfloor \frac{m * K}{K} \right\rfloor \\ &= m. \end{aligned}$$

The data user is allowed to decrypt the embedded database directly and correctly if he/she possesses the encryption key. The process is as follows:

$$\begin{aligned} \text{Dec}(x'', K) &= \left\lfloor \frac{x''}{K} \right\rfloor \\ &= \left\lfloor \frac{m * K + r \pm 1}{K} \right\rfloor \\ &= \left\lfloor \frac{m * K}{K} \right\rfloor \\ &= m. \end{aligned}$$

Watermark extraction: We now prove the correctness of the extracted watermark. The process is as follows:

$$\begin{aligned} w &= \left\lfloor \frac{p'^e}{2} \right\rfloor \\ &= \begin{cases} \left\lfloor \frac{2 * p^{ne} + 1}{2} \right\rfloor, & p'^e = 2 * p^{ne} + 1, \\ \left\lfloor \frac{2 * p^{ne} + 0}{2} \right\rfloor, & p'^e = 2 * p^{ne} + 0, \end{cases} \\ &= \begin{cases} 1, & p'^e = 2 * p^{ne} + 1. \\ 0, & p'^e = 2 * p^{ne} + 0. \end{cases} \end{aligned} \tag{14}$$

6.3 Security Analysis

Based on (z, n) -window one-wayness security model [6], we define a security experiment $\text{Exp}_{\mathcal{A}, \Pi}^{\text{wow}}$ for order-preserving encryption-based watermarking scheme. Suppose that $\Pi = (\text{KGen}, \text{DEnc}, \text{DPre}, \text{WEmb}, \text{WExt}, \text{DDec})$ is an OPEW scheme, λ is a security parameter. For any **probabilistic polynomial-time (PPT)** adversary $\mathcal{A}$, $\text{Exp}_{\mathcal{A}, \Pi}^{\text{ow}}$ is as follows:

- (1) Setup: Run $\text{KGen}(\lambda)$ to generate a key K for OPE scheme and another key K_s for database preprocessing.
- (2) Execution: Run $\text{DEnc}(D, K)$ to generate a encrypted dataset $X = \{x_1, \dots, x_n\}$.
- (3) Execution: Run $\text{DPre}(X, K_s)$ to obtain a preprocessed X .
- (4) Challenge: Run $\text{WEmb}(X, W)$ to obtain embedding watermarks $X'' = (x_1'', \dots, x_n'')$.
- (5) Guess: Given X'' , the adversary outputs an interval (x_L, x_r) .
- (6) Output: If there exists $x_i \in X$ so $x_i \in [x_L, x_R]$ and $x_R - x_L < z$, then $\text{Exp}_{\mathcal{A}, \Pi}^{\text{ow}}$ outputs 1 and 0 otherwise.

THEOREM 6.1. *Our OPEW is secure under (z, n) -window one-wayness security model.*

PROOF. The idea of proof is that if $\mathcal{A}$ can break our scheme, then there exists a PPT adversary $\mathcal{A}'$ breaking OPE scheme proposed by Dyer [13].

Assume that $\mathcal{A}$ breaks the OPEW with non-negligible probability ϵ . Now $\mathcal{A}'$ can construct an interval for OPE. Given a attribute value x_1 , the embedding watermark is $x_1'' = K * x_1 + r_1 \pm 1$ or $x_1'' = K * x_1 + r_1$. $\mathcal{A}'$ can construct a set of ciphertext $\{x_1'' \pm 1, \dots, x_n'' \pm 1\}$ and output the interval $[x_L, x_R]$. From algorithm Enc, it follows that $x_i \in [x_L, x_R]$. In summary, $\mathcal{A}'$ breaks OPE with probability of at least $\frac{\epsilon}{2n}$. But it goes against the security of OPE in Reference [13]. Therefore, any probabilistic polynomial-time $\mathcal{A}$ cannot break the security of the scheme with non-negligible probability. OPEW is secure under (z, n) -window one-wayness security model. $\square$

7 PERFORMANCE ANALYSIS

In this section, we analyze the data utility, database watermark robustness, and preprocessing efficiency by experimental results. We use model analysis and experimental results to evaluate the data utility. In addition, we calculate the bit error rate to analyze to watermark robustness. We also compare the preprocessing of grouping and K-means in terms of the relative embedding capacity, computational costs, and robustness in resisting the modification attack.

Given that to our best knowledge, our method is the first encrypted domain-reversible database watermarking method, we only compare some state-of-the-art reversible database watermarking schemes in the plaintext domain, such as GAHSW [25], DEW [22], and PEEW [16]. The experiments were run on a 3.90 GHz CPU and an 8 GB RAM Intel Core i3 computer. To enhance the universality of our algorithm, we conducted experiments on three datasets, including Heart Disease,³ 3D Road Network,⁴ and Covertypes.⁵ The length of the watermark information was set to 16.

7.1 Data Utility Analysis

The data utility indicates that the embedded data are order-preserving and the decrypted data are lossless. We use model analysis and experimental results to evaluate the data utility.

³<http://archive.ics.uci.edu/ml/datasets/Heart+Disease>.

⁴<http://archive.ics.uci.edu/ml/datasets/3D+Road+Network+%28North+Jutland%2C+Denmark%29>.

⁵<http://archive.ics.uci.edu/ml/datasets/Covertime>.

Table 4. The Mean and Variance of the Processed Database

Attribute name	original database		DEW		PEEW		GAHSW		OPEW	
	Mean	Variance	Mean	Variance	Mean	Variance	Mean	Variance	Mean	Variance
Aspect	141.854	11,757.283	141.614	11,793.525	142.513	12,056.698	141.801	11,772.033	141.854	11,757.283
Elevation	2,867.552	30,170.995	2,867.562	30,170.985	2,867.572	30,170.995	2,867.559	30,201.226	2,867.552	30,170.995
Hillshade_3pm	139.768	860.190	139.677	868.421	140.086	878.411	139.78	863.301	139.768	860.190
Hillshade_9am	218.274	432.806	218.203	445.843	218.491	484.968	218.351	433.972	218.274	432.806
Hillshade_Noon	225.413	213.627	225.499	214.915	225.413	213.627	225.47	214.651	225.413	213.627
H_D_To_Fire_Points	3,184.821	3,049,514.793	3,182.716	3,066,585.861	3,184.821	3,049,514.793	3,184.782	3,049,839.611	3,184.821	3,049,514.793
H_D_To_Hydrology	236.588	36,086.701	236.608	36,292.155	237.852	36,365.727	236.588	37,796.386	236.509	36,101.140
H_D_To_Roadways	3,351.21	2,816,153.860	3,353.614	2,830,210.676	3,360.865	2,828,424.331	3,351.21	2,816,153.860	3,351.207	2,816,492.854

In our scheme, according to the keys of the data users, there are four modes of database access:

- (1) When a data user has neither a watermarking key nor an encryption key, the user can only perform some comparison and sorting operations on the encrypted data of the database.
- (2) When a data user has only the watermarking key, the user can perform copyright verification but cannot recover the original plaintext data through decryption.
- (3) When a data user has only the encryption key, the user can obtain the original data but cannot track the source of the data.
- (4) When a data user has both the encryption key and the watermarking key, the user can not only obtain the original plaintext data but can also track the source of the data.

In cases (1) and (2), the embedded data or the encrypted data still support operations such as attribute value comparison and sorting. In case (4) the embedded data is the same as the original data after extraction and decryption. Therefore, in the experiments, we just compare the data compared the data that is decrypted with the watermark unextracted (3) with the data processed by state-of-the-art plaintext domain database watermarking schemes and the original data. We think this kind of comparison is fair, because the watermark information is still in the plaintext of the data.

The test database is the Coverttype dataset. The mean and variance of eight attributes are used to evaluate the lossless. DEW [22], PEEW [16], and GAHSW [25] are compared with OPEW under the same evaluation terms. The experimental results are given as an average over 10 simulation experiments. Table 4 shows that the mean and variance of DEW, PEEW, and GAHSW are shifted by different degrees compared to the original database attribute, and the mean and variance of each attribute of OPEW are the same as original database attribute.

In addition, we evaluated the impact of the OPEW method on classification. We conducted evaluation experiments on the 3D Road Network dataset provided by the University of California, which contains 434,874 instances, each with four attributes. Four kinds of classifiers, KNN, SVM, DT, and NB, are used to classify the attribute of “class,” which has three labels. The results are shown in Table 5. From the table, we can see that both OPE and OPEW change the classification accuracy of each class to a certain degree; however, the accuracies of OPE and OPEW on each class are identical. This is because OPE introduces a certain degree of randomness into the data, whereas the randomness that is introduced by OPEW based on OPE is very slight and negligible.

7.2 Database Watermark Robustness Analysis

When analyzing data robustness, the test database was the Coverttype dataset. The dataset contains 581,012 tuples and 54 attributes. Experiments were conducted on 1,000 selected tuples and 10 selected numeric attributes. The database modification, deletion, and addition attacks were used to test the robustness of the database. We evaluated the robustness by the **bit error rate (BER)**: the ratio of the number of error extraction bits to the number of watermark bits. The formula for

Table 5. OPE's Impact on Various Categorical Measurements (Accuracy Ac)

classifier	classes	$Ac_{Original}$	Ac_{OPE}	Ac_{OPEW}
KNN	1	0.75	1.0	1.0
	2	0.744	0.75	0.75
	3	1.0	0.744	0.744
SVM	1	0.6661	0.15	0.15
	2	0.2	0.7	0.7
	3	0.5	0.5454	0.5454
DT	1	0.917	1.0	1.0
	2	1.0	0.917	0.917
	3	0.8	0.8	0.8
NB	1	1.0	0.77	0.77
	2	0.125	1.0	1.0
	3	0.77	0.125	0.125

calculating the BER is as follows:

$$BER = \frac{\sum_{i=1}^{N_g} w_i \oplus w_i^{det}}{N_g}, \quad (15)$$

where w_i is an embedded watermark bit and w_i^{det} is the modified watermark bit. We can see that a lower value of the BER means higher watermarking robustness.

In the database watermark attack experiment, we gradually increased the attack level from 0%, 10%, and 20% to the final 90% of the tuples. Under such conditions, we compared the results of OPEW with those of DEW, PEEW, and GAHSW. We conducted the experiments 50 times, then averaged the experimental results and plotted them. The results for modification and deletion are shown in Figures 4 and 5, respectively. In the two figures, the x-axis represents the ratio of the number of tuples being attacked to the total number of tuples, and the y-axis is the BER.

For the addition attack, we randomly added some tuples to the database and observed the correctness of the extraction of the watermark by the copyright owner. After randomly inserting the tuples, we found that the watermark can be 100% correctly extracted regardless of how it is added. Therefore, OPEW can resist insertion attacks, as can other methods.

For the modification attack, the attribute values of the tuples in the database are randomly modified, the main purpose of which is to add noise to weaken the watermark signal so the copyright owner cannot correctly extract the watermark information. As the number of modified tuples increases, the BERs of our watermark extraction increase. From Figure 4, we can find that the ability of OPEW to resist the modification attack is higher than that of PEEW and DEW but lower than that of GAHSW.

For the deletion attack, tuples of the database are randomly deleted to destroy the watermark so the copyright owner cannot extract the watermark or extract the wrong watermark information.

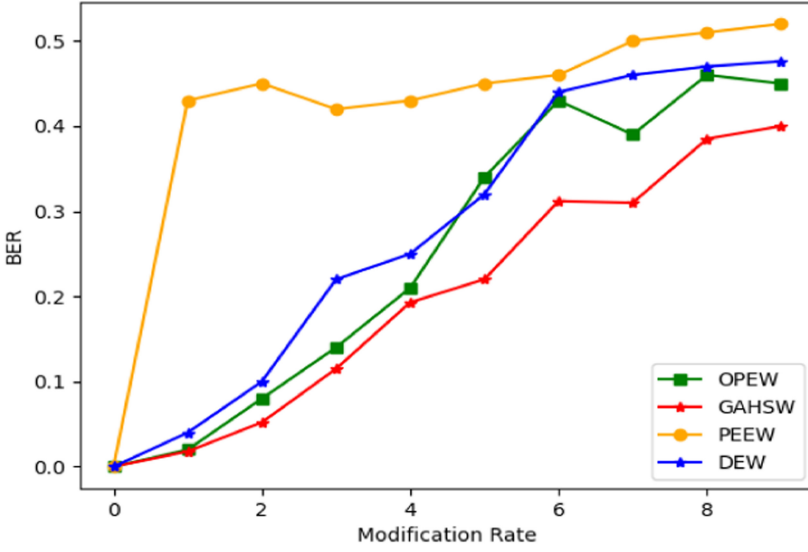


Fig. 4. Different watermarking methods resisting the modification attack.

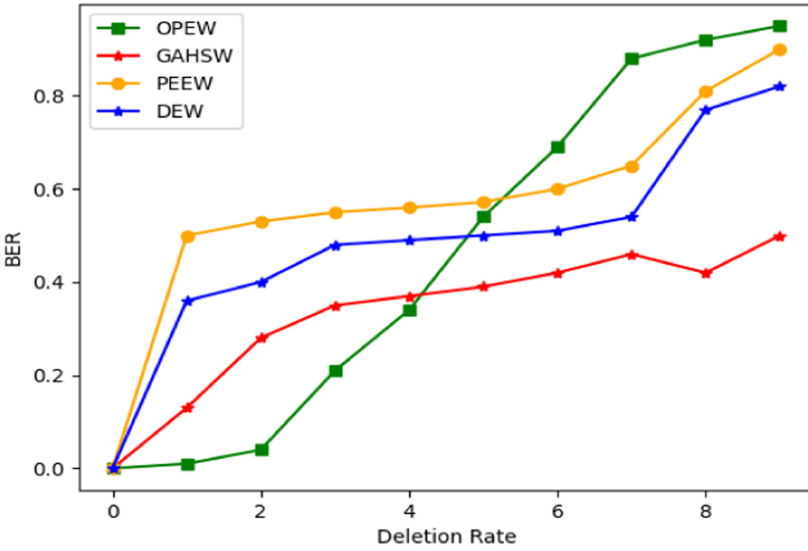


Fig. 5. Different watermarking methods resisting the deletion attack.

As the number of deleted tuples increases, the BERs of our extracted watermarks increase. From Figure 5, we can see that the ability of OPEW to resist the deletion attack is higher than other methods when deletion rate is below 50%. The reason why the robustness of the proposed OPEW is not comparable to other methods is because OPEW is in the encryption domain, where the encrypted and embedded data should retain the order-preserving nature; another reason is that in the preprocessing stage, GAHSW uses a GA to optimize the robustness under the constraint of the embedding capacity, while the proposed OPEW uses K-means based on grouping to improve the robustness, which is not comparable to the GA-based method. In the following, we analyze the advantages and disadvantages of K-means-based preprocessing.

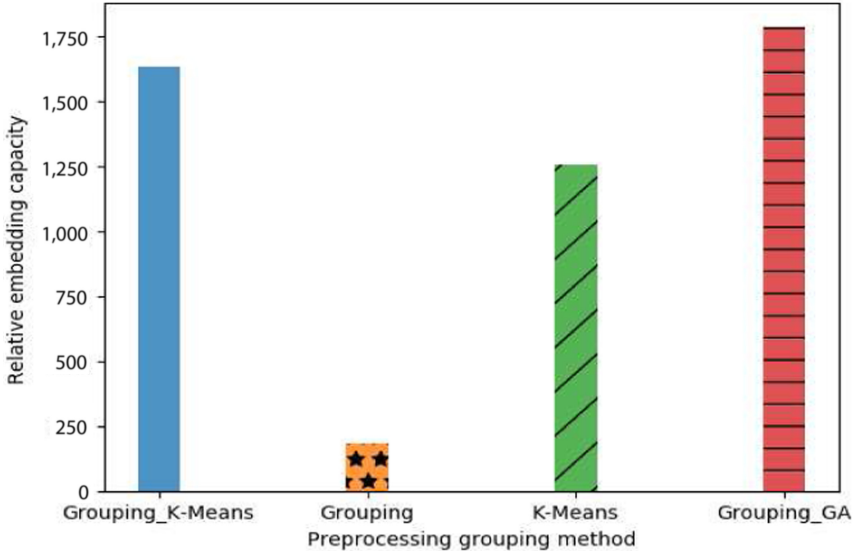


Fig. 6. Comparison of the relative embedding capacities of different preprocessing methods.

7.3 Preprocessing Efficiency Analysis

Grouping is the most common means of improving the robustness and embedding rate of database watermarks. The most common grouping method in database watermarking is to use the formula given in Equation (2) for grouping. We use the K-means algorithm to further improve the robustness. Here, we conduct experiments to show the effectiveness of using the K-means algorithm in improving the robustness. In the first experiment, we compared the relative embedding capacities of the traditional grouping method using Equation (2), our improved grouping method (grouping + K-means), the method only using K-means, and the method of Reference [25] (grouping + GA). The relative embedding capacity is the real embedded number of tuples in each group. The experiments were conducted on the Heart Disease dataset, which contains 18,492 tuples and eight attributes. For the sake of simplicity, we selected 1,000 tuples for the experiments and generated a new attribute as the primary key. We conducted five sets of experiments and averaged the experimental results. The results are shown in Figure 6.

From Figure 6, we can see that the relative embedding capacity of grouping is the poorest, the method using only K-means exhibits considerably improved relative embedding capacity, our improved grouping method (grouping + K-means) achieves the second best performance, and the method of Reference [25] (grouping + GA) is the best. We can give a simple example to analyze why K-means can further improve the relative embedding capacity based on grouping.

For the sake of simplicity, we compare the traditional grouping method with our method of grouping based on K-means. Assume that a group of data obtained after the grouping operation is [2, 3, 4, 4, 5, 7, 8, 8, 8, 9]. The grouping operation will be embedded in position 7 where the peak point is generated and position 7 to the left of the peak point. When further performing the K-means operation, assuming K is 2 in K-means, we can generate two groups [2, 3, 4, 4, 5] and [7, 8, 8, 8, 9], and thus, we can obtain two peak points and have more embedding positions.

We attacked the database from both the modification and deletion perspectives. We plot the BER results under such attacks in Figures 8 and 9. In the figures, the vertical axis is the BER, and the

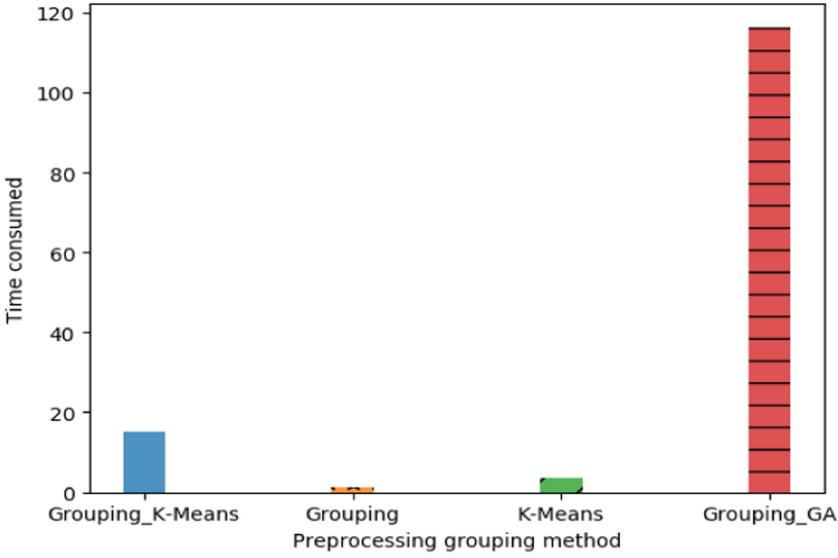


Fig. 7. Time consumption of different pretreatment methods.

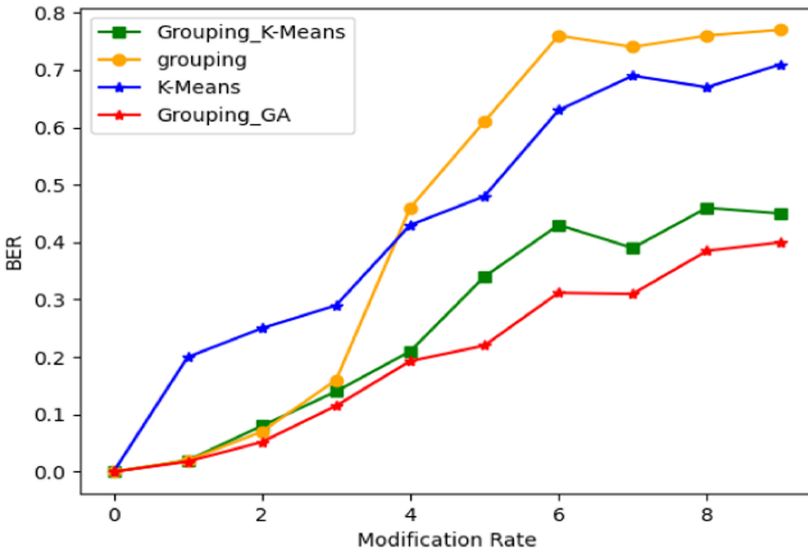


Fig. 8. Different preprocessing methods resisting the modification attack.

horizontal axis represents the percentage of the database that was modified or deleted. From the figures, we can see that the difference in the robustness of the four methods is not very large. By comparison, the robustness of the GA-based method is the best.

The GA-based method is more robust than our method, because the GA-based method can find an optimal solution via a time cost iteration, while the K-means-based solution may not be the optimal one. However, the K-means-based method is more efficient than the GA-based method. Here, we further analyze the time cost of each preprocessing method. We conducted five sets of experiments and calculated the average time cost of each preprocessing method. The results are shown in Figure 7, from where we can see that the time cost of the GA-based method is the

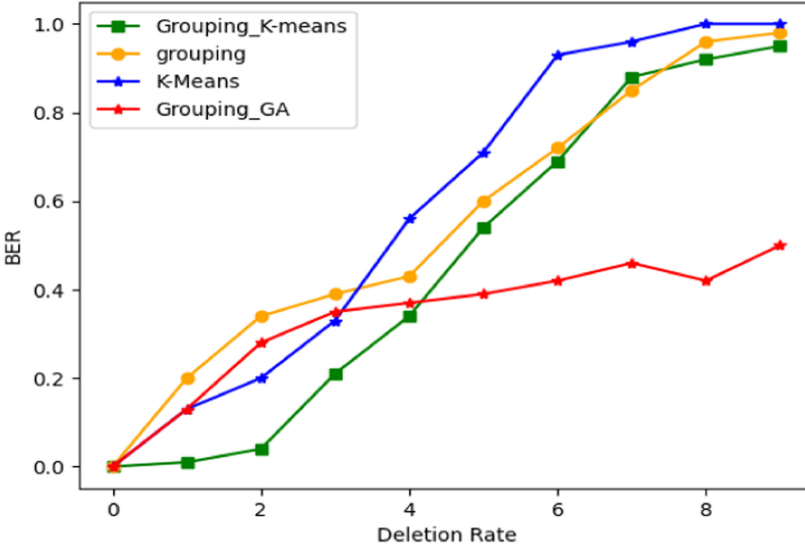


Fig. 9. Different preprocessing methods resisting the deletion attack.

highest—approximately 10 times that of the K-means-based method. Therefore, considering both robustness and efficiency, the K-means-based method has advantages in big data application scenarios.

8 DISCUSSION

OPE and ORE: In our scheme, we adopt OPE scheme [13] to preserve database privacy. The OPE scheme has proved security under (z, n) -window one-wayness [6] security model. Therefore, any PPT adversary cannot obtain information about plaintext location. **Order Revealing Encryption technology (ORE)** [7, 8, 32] is a generalization of OPE that allows for stronger security. The order of ciphertext in ORE is determined by a publicly computable comparison function. It is possible for ORE to achieve **indistinguishability under an ordered chosen plaintext attack (IND-OCPA)**. During watermark embedding phase in OPEW, the data owner needs to use Equation (3) to predict values in each group and then use these errors to construct a frequency histogram. It requires the numerical ciphertext. In addition, during the watermark extraction phase, the data user needs to cluster the data into subgroups with K-means technology, which requires that the ciphertext keep the order-preserving property. It puts a high requirement on the computing power of the data user. In the future, we can introduce a trusted proxy to share some of the computational burdens so we can use ORE to improve the security of our scheme.

Queries: The scheme supports the order, comparison, maximum, and minimum queries according to the value of embedded data. But the user cannot carry out any kind of query with the encryption key and watermark key. The reason for this limitation is that we only adopt OPE encryption. The OPE introduces a random number; it cannot be used to process equality queries. More importantly, our scheme focuses on providing privacy protection and copyright protection for the database. We can use symmetric searchable encryption [31] to achieve equality conjunction queries and secure join [36] to support equi-join queries between two tables. (We can adopt DET and JOIN encryption [34] to perform equality checks and equi-join queries, but inference attacks in Reference [33] can recover the plaintext from DET database columns using the encrypted column and publicly available auxiliary information.) We can use the two ancillary columns c-LSH

and c-BF [26] to achieve wildcard keyword queries and fuzzy keyword queries on encrypted database. It does not affect the final query results; the watermark can be extracted if the query results contain complete attribute columns involved with watermark embedding. Also, it has no effect on decryption if the correct decryption key is provided.

OPEW algorithm: The database processing phase is more crucial. It is more close to robustness. Grouping and Clustering enhance the embedding capability of the data. So even if a watermark is broken, the data user can use other embedded data to recover the watermark. In addition, the watermark embedding phase is dependent on the encrypted database after the OPE encryption. We shrink the interval of r to provide the redundant space for embedding watermark.

Side-channel attack: Aria et al. [37] showed a cache side-channel attack on SQLite that obtains approximate volumes of range queries made to an encrypted database. Boenisch et al. [4] utilized the implicit conditional logic of the database runtime optimization to launch side-channel attacks. For the cache side-channel attack, we can adopt the real-time detection method [9]. For power attacks, we can use the digital VLSI design flow [45]. Also, we can use register configuration to send random pseudo operation insertion requests to some idle modules inside the CPU at a certain frequency to weaken the link between the data and the power consumption.

9 CONCLUSION

This article proposes an encryption domain lossless database watermarking scheme based on OPE and reversible watermarking, providing both data privacy protection and copyright protection. We first perform order-preserving encryption to generate redundant space and then design the watermark embedding algorithm without breaking the encryption of the preserved order. In the watermark embedding process, the reversible watermarking scheme HS-PEE is used to reduce the modification degree and improve the embedding capacity. The method is lossless, and embedded data can be correctly recovered to their original plaintext regardless of whether the embedded watermark has been extracted. Theoretical analysis and experimental results show that the proposed scheme can retain the nature of OPE, achieve good performance in terms of embedding capacity, maintain high anti-attack robustness, and incur a relatively low computational cost. In the future, privacy protection and copyright tracking methods with better performance deserves further investigation. Considering more complex applications in big data-related scenarios, we will also investigate the adaption of our method on non-relational database.

REFERENCES

- [1] Rakesh Agrawal and Jerry Kiernan. 2002. Watermarking relational databases. In *Proceedings of the 28th International Conference on Very Large Databases*. Elsevier, 155–166.
- [2] Rakesh Agrawal, Jerry Kiernan, Ramakrishnan Srikant, and Yirong Xu. 2004. Order preserving encryption for numeric data. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. ACM, 563–574.
- [3] Elisa Bertino, Beng Chin Ooi, Yanjiang Yang, and Robert H. Deng. 2005. Privacy and ownership preserving of outsourced medical data. In *Proceedings of the 21st International Conference on Data Engineering (ICDE'05)*. IEEE, 521–532.
- [4] Franziska Boenisch, Reinhard Munz, Marcel Tiepelt, Simon Hanisch, Christiane Kuhn, and Paul Francis. 2021. Side-channel attacks on query-based data anonymization. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, 1254–1265.
- [5] Alexandra Boldyreva, Nathan Chenette, Younho Lee, and Adam O'Neill. 2009. Order-preserving symmetric encryption. In *Proceedings of the Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 224–241.
- [6] Alexandra Boldyreva, Nathan Chenette, and Adam O'Neill. 2011. Order-preserving encryption revisited: Improved security analysis and alternative solutions. In *Proceedings of the Annual Cryptology Conference*. Springer, 578–595.
- [7] Dan Boneh, Kevin Lewi, Mariana Raykova, Amit Sahai, Mark Zhandry, and Joe Zimmerman. 2015. Semantically secure order-revealing encryption: Multi-input functional encryption without obfuscation. In *Proceedings of the Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 563–594.

- [8] Nathan Chenette, Kevin Lewi, Stephen A. Weis, and David J. Wu. 2016. Practical order-revealing encryption with limited leakage. In *Fast Software Encryption*, Thomas Peyrin (Ed.). Springer, Berlin, 474–493.
- [9] Jonghyeon Cho, Taehyun Kim, Soojin Kim, Miok Im, Taehyun Kim, and Youngjoo Shin. 2020. Real-time detection for cache side channel attack using performance counter monitor. *Appl. Sci.* 10, 3 (2020).
- [10] Weiqi Dai, Chunkai Dai, Kim-Kwang Raymond Choo, Changze Cui, Deiqing Zou, and Hai Jin. 2019. SDTE: A secure blockchain-based data trading ecosystem. *IEEE Trans. Inf. Forens. Secur.* 15 (2019), 725–737.
- [11] Roshan Dathathri, Blagovesta Kostova, Olli Saarikivi, Wei Dai, Kim Laine, and Madan Musuvathi. 2020. EVA: An encrypted vector arithmetic language and compiler for efficient homomorphic computation. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. Association for Computing Machinery, 546–561.
- [12] Roshan Dathathri, Olli Saarikivi, Hao Chen, Kim Laine, Kristin Lauter, Saeed Maleki, Madanlal Musuvathi, and Todd Mytkowicz. 2019. CHET: An optimizing compiler for fully-homomorphic neural-network inferencing. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*. Association for Computing Machinery, 142–156.
- [13] James Dyer, Martin Dyer, and Jie Xu. 2017. Order-preserving encryption using approximate integer common divisors. In *Data Privacy Management, Cryptocurrencies and Blockchain Technology*. Springer, 257–274.
- [14] Liang Fan, Yu Wei, An Dou, Qingyu Yang, Xinwen Fu, and Zhao Wei. 2018. A survey on big data market: Pricing, trading and protection. *IEEE Access* 6, 99 (2018), 15132–15154.
- [15] Mahmoud E. Farfoura and Shi-Jinn Horng. 2010. A novel blind reversible method for watermarking relational databases. In *Proceedings of the International Symposium on Parallel and Distributed Processing with Applications*. IEEE, 563–569.
- [16] Mahmoud E. Farfoura, Shi-Jinn Horng, and Xian Wang. 2013. A novel blind reversible method for watermarking relational databases. *J. Chinese Instit. Eng.* 36, 1 (2013), 87–97.
- [17] Javier Franco-Contreras, Gouenou Coatrieux, Frederic Cuppens, Nora Cuppens-Boulahia, and Christian Roux. 2017. Robust lossless watermarking of relational databases based on circular histogram modulation. *IEEE Trans. Inf. Forens. Secur.* 9, 3 (2017), 397–410.
- [18] Jun Furukawa. 2013. Request-based comparable encryption. In *Proceedings of the European Symposium on Research in Computer Security*. Springer, 129–146.
- [19] Craig Gentry et al. 2009. *A Fully Homomorphic Encryption Scheme*, Vol. 20. Stanford University, Stanford, CA.
- [20] David Gross-Amblard. 2011. Query-preserving watermarking of relational databases and XML documents. *ACM Trans. Datab. Syst.* 36, 1 (2011), 1–24.
- [21] Fei Guo, Jianmin Wang, and Deyi Li. 2006. Fingerprinting relational databases. In *Proceedings of the ACM Symposium on Applied Computing*. 487–492.
- [22] Gaurav Gupta and Josef Pieprzyk. 2008. Reversible and blind database watermarking using difference expansion. In *Proceedings of the 1st International Conference on Forensic Applications and Techniques in Telecommunications, Information, and Multimedia and Workshop*, Vol. 24, 1–6.
- [23] Saransh Gupta and Tajana Šimunić Rosing. 2021. Invited: Accelerating fully homomorphic encryption with processing in memory. In *Proceedings of the 58th ACM/IEEE Design Automation Conference (DAC)*. 1335–1338. DOI: <https://doi.org/10.1109/DAC18074.2021.9586285>
- [24] Nick Howgrave-Graham. 2001. Approximate integer common divisors. In *Proceedings of the International Cryptography and Lattices Conference*. Springer, 51–66.
- [25] Donghui Hu, Dan Zhao, and Shuli Zheng. 2018. A new robust approach for reversible database watermarking with distortion control. *IEEE Trans. Knowl. Data Eng.* 31, 6 (2018), 1024–1037.
- [26] Jiaxun Hua, Yu Liu, He Chen, Xiuxia Tian, and Cheqing Jin. 2020. An enhanced wildcard-based fuzzy searching scheme in encrypted databases. *World Wide Web* 23, 3 (2020), 2185–2214.
- [27] Lei Jiang, Qian Lou, and Nrushad Joshi. 2022. MATCHA: A fast and energy-efficient accelerator for fully homomorphic encryption over the torus. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. Association for Computing Machinery, 235–240.
- [28] Tapas Kanungo, David M. Mount, Nathan S. Netanyahu, Christine D. Piatko, Ruth Silverman, and Angela Y. Wu. 2002. An efficient k-means clustering algorithm: Analysis and implementation. *IEEE Trans. Pattern Anal. Mach. Intell.* 7 (2002), 881–892.
- [29] Meng Li, Yifei Chen, Shuli Zheng, Donghui Hu, Chhagan Lal, and Mauro Conti. 2020. Privacy-preserving navigation supporting similar queries in vehicular networks. *IEEE Trans. Depend. Secure Comput.* 99 (2020), 1–1.
- [30] Meng Li, Yifei Chen, Liehaung Zhu, Zijian Zhang, Jianbing Ni, Chhagan Lal, and Mauro Conti. 2022. Astraea: Anonymous and secure auditing based on private smart contracts for donation systems. *IEEE Trans. Depend. Secure Comput.* (2022), 1–17.

- [31] Weipeng Lin, Ke Wang, Zhilin Zhang, Ada Waichee Fu, Raymond Chi-Wing Wong, Cheng Long, and Chunyan Miao. 2022. Towards secure and efficient equality conjunction search over outsourced databases. *IEEE Trans. Cloud Comput.* 10, 2 (2022), 1445–1461.
- [32] Zheli Liu, Siyi Lv, Jin Li, Yanyu Huang, Liang Guo, Yali Yuan, and Changyu Dong. 2022. EncodeORE: Reducing leakage and preserving practicality in order-revealing encryption. *IEEE Trans. Depend. Secure Comput.* 19, 3 (2022), 1579–1591. DOI: <https://doi.org/10.1109/TDSC.2020.3029845>
- [33] Muhammad Naveed, Seny Kamara, and Charles V. Wright. 2015. Inference attacks on property-preserving encrypted databases. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS'15)*. Association for Computing Machinery, New York, NY, 644–655.
- [34] Raluca Ada Popa, Catherine M. S. Redfield, Nickolai Zeldovich, and Hari Balakrishnan. 2011. CryptDB: Protecting confidentiality with encrypted query processing. In *Proceedings of the 23rd ACM Symposium on Operating Systems Principles*. Association for Computing Machinery, 85–100.
- [35] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Srinivas Devadas, Ronald Dreslinski, Christopher Peikert, and Daniel Sanchez. 2021. F1: A fast and programmable accelerator for fully homomorphic encryption. In *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture*. Association for Computing Machinery, 238–252.
- [36] Masoumeh Shafieinejad, Suraj Gupta, Jin Yang Liu, Koray Karabina, and Florian Kerschbaum. 2022. Equi-joins over encrypted data for series of queries. In *Proceedings of the IEEE 38th International Conference on Data Engineering (ICDE)*. 1635–1648.
- [37] Aria Shahverdi, Mohammad Shirinov, and Dana Dachman-Soled. 2021. Database reconstruction from noisy volumes: A cache side-channel attack on SQLite. In *Proceedings of the 30th USENIX Security Symposium (USENIX Security'21)*. USENIX Association, 1019–1035.
- [38] Yun Q. Shi. 2006. Reversible data hiding. *IEEE Trans. Circ. Syst. Vid. Technol.* 16, 3 (2006), 354–362.
- [39] Yun Q. Shi, Zhicheng Ni, Dekun Zou, Changyin Liang, and Guorong Xuan. 2004. Lossless data hiding: Fundamentals, algorithms and applications. In *Proceedings of the IEEE International Symposium on Circuits and Systems*. IEEE, II–33.
- [40] Sujoy Sinha Roy, Furkan Turan, Kimmo Jarvinen, Frederik Vercauteren, and Ingrid Verbauwhede. 2019. FPGA-based high-performance parallel architecture for homomorphic computing on encrypted data. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA'19)*. 387–398. DOI: <https://doi.org/10.1109/HPCA.2019.00052>
- [41] Radu Sion, Mikhail Atallah, and Sunil Prabhakar. 2004. Rights protection for relational data. *IEEE Trans. Knowl. Data Eng.* 16, 12 (2004), 1509–1525.
- [42] Isamu Teranishi, Moti Yung, and Tal Malkin. 2014. Order-preserving encryption secure beyond one-wayness. In *Proceedings of the International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 42–61.
- [43] Diljith M. Thodi and Jeffrey J. Rodriguez. 2007. Expansion embedding techniques for reversible watermarking. *IEEE Trans. Image Process.* 16, 3 (2007), 721–730.
- [44] Jun Tian. 2003. Reversible data embedding using a difference expansion. *IEEE Trans. Circ. Syst. Vid. Technol.* 13, 8 (2003), 890–896.
- [45] K. Tiri and I. Verbauwhede. 2005. A VLSI design flow for secure side-channel attack resistant ICs. In *Proceedings of the Design, Automation and Test in Europe Conference*. 58–63.
- [46] Shijun Xiang and Jiayong He. 2017. Database authentication watermarking scheme in encrypted domain. *IET Inf. Secur.* 12, 1 (2017), 42–51.
- [47] Xinpeng Zhang. 2011. Reversible data hiding in encrypted image. *IEEE Sig. Process. Lett.* 18, 4 (2011), 255–258.
- [48] Xinpeng Zhang. 2011. Separable reversible data hiding in encrypted image. *IEEE Trans. Inf. Forens. Secur.* 7, 2 (2011), 826–832.
- [49] Yong Zhang, Bian Yang, and Xia-Mu Niu. 2006. Reversible watermarking for relational database authentication. *J. Comput.* 17, 2 (2006), 59–66.

Received 6 June 2021; revised 15 November 2022; accepted 27 March 2023